

项目 API 接入文档

2026-09-14 · 按当前代码核对 · Assistant 0.3.6 / RAG 0.5.1

本文面向页面开发者与服务调用方，介绍现有 HTTP 接口、请求与返回约定。当前主要入口为 Agent V2，支持政策问答、设备和已有生鲜价格查询。轨迹、资费、时限当前显示暂不可用；调用方按能力目录和服务端响应处理。

1 先选对服务与入口

接口面	路径与用途	接入方式
Agent V2（主要入口）	/v2/agent/... 理解、补槽、候选选择、会话恢复	浏览器经同源 /api 前缀；可信服务直连 Assistant。
Assistant V1（兼容）	/v1/chat 显式选择政策或设备价格，单轮查询	使用 Assistant 服务 Key；不带 Agent 会话语义。
政策 RAG（独立服务）	/v1/retrieve、/v1/chat 直接检索政策或生成引用式回答	单独的 RAG 地址和 Key；不是 Assistant 的 /v1/chat。

当前受控 Web 入口只代理下面的 Agent V2 业务路径。它不开放 RAG、V1 兼容路由、readiness、metrics 或 Swagger。服务代码中存在接口，不等于该接口能通过当前网页端口访问。

方法	浏览器同源路径	正常返回
POST	/api/v2/agent/browser-session	200 · 身份引用与到期时间
GET	/api/v2/agent/capabilities	200 · 能力数组
POST	/api/v2/agent/messages	200 · JSON 或 SSE
GET	/api/v2/agent/conversations/{id}	200 · 最新持久停止点
DELETE	/api/v2/agent/conversations/{id}	204 · 空响应

GET /capabilities 直接返回数组，各项含 intent、display_name、available、capability_version 与 required_inputs。版本可为 null；required_inputs 是静态能力描述，当前实际补槽与候选应以消息响应为准，不由客户端预先收齐不可用能力的参数。

通用约定

请求使用 UTF-8 JSON，发送 Content-Type: application/json。查询文本去首尾空白后为 1~2000 字符。{id} 使用服务端返回的小写 UUID，不自行推导。本文的 JSON、令牌、价格、日期和引用均为合成示例，不代表实时业务结果。

X-Request-ID 可选：1~64 个字母、数字、下划线或 ASCII 连字符；非法或缺失时服务生成新值。响应头返回最终 X-Request-ID。它用于排查问题，不替代幂等键或会话身份。

可信服务使用管理员分配的非浏览器代理 Key，发送 Bearer 或 X-API-Key（二选一），直连对应服务；Agent owner 按服务 Key 隔离，不等同于终端用户身份。浏览器 Cookie 为 SameSite=Strict；现有内网 HTTP 例外不提供传输加密。

阅读顺序：浏览器接入 → V2 消息与候选 → 商品结果 → SSE → V1 / RAG → 错误与健康检查。完整 DTO 与字段边界以本文末尾列出的源码、OpenAPI 和生成 TypeScript 为准。

2 浏览器身份与最小调用

页面不保存服务 Key。Nginx 注入服务凭据，后端用签名 HttpOnly Cookie 确定 owner。browser-session 返回 {session_ref, expires_at}；前者为 64 位小写十六进制身份引用，不能替代 Cookie，后者为带时区的到期时间。

请求头 / 身份	何时需要
Cookie + X-Agent-Session	先核验 browser-session；后续能力、消息、快照、删除请求带 Cookie 和返回的 session_ref。
X-Agent-Contract: product-price-v1	当前商品 profile 的浏览器能力、消息和快照请求必需；表示客户端兼容版本，不是认证。
Origin	浏览器 POST / DELETE 由浏览器自动发送，须与配置的网页源精确一致；不要用跨域脚本绕过。
Idempotency-Key	仅 V2 POST /messages 必填。每条新消息生成一次；重试保留原键和原业务内容。

同源页面代码示例

示例位于 apps/chat-web/src/，复用现有 createld；不要假定内网 HTTP 一定支持 crypto.randomUUID。正式组件可直接复用 agent-api.ts 的校验和错误处理。

```
import { createld } from './id'
const base = '/api/v2/agent'
async function call(path, options = {}) {
  const r = await fetch(base + path, {
    credentials: 'same-origin', cache: 'no-store', ...options
  })
  if (!r.ok) throw { status: r.status,
    requestId: r.headers.get('X-Request-ID'), body: await r.text() }
  return r.json()
}
const identity = await call('/browser-session', {
  method: 'POST', headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ reset: false })
})
const headers = {
  'X-Agent-Session': identity.session_ref,
  'X-Agent-Contract': 'product-price-v1'
}
const catalog = await call('/capabilities', { headers })
if (!catalog.some(x => x.intent === 'product_price' && x.available))
  throw new Error('商品价格查询暂不可用')
const key = createld() // 在发送前保存；重试不重新生成
const payload = { message: 'iPhone 17 256GB 多少钱',
  explicit_intent: 'product_price', stream: false }
const result = await call('/messages', {
  method: 'POST', headers: { ...headers,
    'Content-Type': 'application/json', 'Idempotency-Key': key },
  body: JSON.stringify(payload)
})
```

无 Cookie 时 reset:false 可建立身份；有效 Cookie 只核验，不续期、不重新发放。失效后由用户明确选择 reset:true 重建身份；旧 pending 请求不自动换身份重发。刷新时先核验身份，再恢复历史或读取快照。

3 Agent V2：发送、恢复与读取

POST /v2/agent/messages

浏览器加 /api 前缀。Agent V2 的 stream 默认 false；与 V1 / RAG 的默认 true 不同。Idempotency-Key 必填，长度 1~128，仅允许可见且无空白的 ASCII 字符。建议使用随机 UUID，不放业务数据。

请求字段	类型 / 默认值	约束与含义
conversation_id	UUID / null	null 或省略为新会话；继续时使用上次响应的 ID。
message	string / null	新会话必填；后续补槽用自然语言或控件序列化文本。
explicit_intent	意图 / null	可选的明确入口；不得为 unknown。仅已有澄清任务可只传意图。
confirm_overwrite	boolean / false	确认当前字段冲突；不是通用业务审批或资费范围确认。
stream	boolean / false	false 为 JSON；true 为 SSE。不会改变业务幂等指纹。
price_selection	object / null	仅已有会话的商品候选选择，下一节给出独立请求。

```
{
  "message": "上海黄瓜零售均价多少钱一斤",
  "explicit_intent": "product_price",
  "stream": false
}
```

当前能力目录为 policy、product_price、tracking、delivery_time、postage。available 表示已装配，不保证上游在线。catalog_v2 不再接受新请求显式指定 device_price，会返回 409 agent_price_intent_upgraded；保留旧历史的读取和渲染。

请求只接受上表字段。不要提交 history、owner_id、工具名、SQL 或 Graph State。required_inputs.name 是 UI 描述；conditions.region_text、mail_no 等不是请求顶层字段。普通补槽应复用 agent-slot-input.ts 的序列化规则写入 message。

消息幂等与主动新查询

每次补充、确认或选择都是一条新消息，使用新键。网络结果不确定时，保留原会话字段、键与业务内容重试；同键不同内容返回冲突。同一已完成消息可在 JSON / SSE 间重放，conversation_id / turn_id 与事实保持一致，request_id 可不同。主动再次查询使用新键和 message，会重新取数；这不等同于重放旧结果。

GET / DELETE /v2/agent/conversations/{id}

操作	语义
GET → 200 AgentResponse	读取该 owner 的最新停止态；不执行 Graph / 模型 / Tool，不刷新数据或候选期限。不是会话列表或全量历史接口。
DELETE → 204 空响应	清理该会话的状态和收据，保留必要防复活记录；同 owner 重复删除可返回 204。不存在或非本人会话返回 404。

会话执行中会返回 409，而不是并发推进。存在未确认请求时，不应以较早快照覆盖它；会话过期或身份改变后，不应自动换 ID 重发。TTL 取决于服务端配置，读取不续期。

4 Agent V2：响应、补槽与候选

200 响应为 AgentResponse；它描述一次工作流的停止点，不保证查询有结果。result 仅在 completed 时出现，failure 仅在 failed 时出现。reply 是可显示文本，required_inputs 与 result 才是控件和事实卡片的数据来源。

phase / next_action	客户端处理
waiting_user / collect_slots	按 required_inputs 收集缺项、确认冲突或选择候选。
waiting_user / clarify_intent	展示服务端意图选项，用 explicit_intent 恢复。
completed / complete	按 result.type 和 status 渲染；success / partial / no_match 分开。
handoff / handoff ; failed / failed	前者没有可执行路径，非人工工单；后者显示 failure 与安全提示。

完整候选停止态示例（合成）

```
{
  "request_id": "api-demo-001",
  "conversation_id": "11111111-1111-4111-8111-111111111111",
  "turn_id": "22222222-2222-4222-8222-222222222222",
  "phase": "waiting_user", "intent": "product_price",
  "reply": "请选择需要查询的商品规格。",
  "next_action": "collect_slots",
  "required_inputs": [{
    "name": "price_selection", "label": "商品选择", "type": "choice",
    "validation_hint": "请选择当前候选", "choices": ["1. 示例设备 256GB"],
    "price_candidates": [{
      "candidate_token": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa",
      "label": "1. 示例设备 256GB", "expires_at": "2026-09-14T10:10:00+08:00"
    }]
  }],
  "result": null, "failure": null, "warnings": []
}
```

选择候选：同一 /messages 接口，使用新幂等键

```
{
  "conversation_id": "11111111-1111-4111-8111-111111111111",
  "price_selection": {
    "candidate_token": "aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa"
  },
  "stream": false
}
```

candidate_token 为服务端返回的 43 字符 base64url 不透明令牌。示例令牌仅用于展示，实际必须原样使用当前响应值；不得提交数据库 ID、猜测令牌，或同时添加 message、explicit_intent、confirm_overwrite:true。选择后按同一商品身份重读当前价格，可能与发现候选时不同。

每个候选列表最多 20 项，expires_at 为绝对期限，最长不超过 10 分钟且受会话期限约束。刷新、恢复和重放不续期；改型号、地区或口径可能使候选失效。商品查询受 2 次逻辑调用 / 1 次重试 / 3 轮澄清预算限制，不通过刷新重新获得额度。

5 商品价格结果：金额、单位与来源

完成报价时 result.type=product_price，result.data 使用下述公开结构。外层仍为上一节的 AgentResponse，next_action=complete；status 可为 success 或 partial。无匹配不返回虚构 items。

result.data 完整示例（合成的生鲜单位价）

```
{
  "type": "product_price", "schema_version": "1", "truncated": false,
  "items": [{
    "identity": {
      "kind": "fresh", "commodity_name": "黄瓜",
      "source_specification": "散装", "market_name": "示例市场"
    },
    "quote": {
      "kind": "priced", "currency": "CNY", "current_price": "3.50",
      "original_price": null, "original_price_type": "NONE",
      "availability": "ON_SALE", "price_nature": "RETAIL_AVERAGE",
      "quoted_unit": "CNY_PER_500G",
      "unit_price": { "amount": "7.000000", "unit": "CNY_PER_KG" },
      "observed_at": "2026-09-13T00:00:00+08:00", "time_precision": "DAY"
    },
    "source": {
      "name": "示例零售来源", "profile": "DEMO_RETAIL",
      "url": "https://example.invalid/price"
    },
    "region": { "scope": "CITY", "code": "310100" },
    "queried_at": "2026-09-14T10:00:00+08:00",
    "freshness": "unknown", "last_known_price": null
  }]
}
```

字段 / 分支	客户端必须保留的语义
identity.kind	device：brand、product_name、specification 对象；fresh：commodity_name、source_specification、market_name。
quote.kind / 金额	priced 才有正数 current_price；availability_only 表示缺货、下架或即将上市等无金额状态，不能补成 0。金额均为字符串。
price_nature / 单位	设备 RETAIL_OFFER 按配置 / 件；生鲜为 RETAIL_AVERAGE 或 WHOLESALE_AVERAGE。保留原始 quoted_unit 和标准 unit_price。
original_price_type	NONE / CROSSED_OUT / MSRP / EXPLICIT_ORIGINAL。原价、指导价和现价不能混为一谈；不自行推导折扣承诺。
时间与 freshness	observed_at 为来源观察时间，queried_at 为读取时间；time_precision=DAY / INSTANT / UNKNOWN，freshness=unknown / fresh / stale。

元/500g 转元/kg 是精确十进制 ×2；设备报价不能按商品重量任意换算。新读取不代表新价格；当日请求遇到旧来源日可能返回 partial，仍应显示真实数据日。DAY 按北京时间展示来源日，unknown 不标成实时价。

items 为 1~100 项，truncated 表示结果被截断；不同来源逐项展示，不在客户端合成均价。source.url 用于核验来源。last_known_price 是独立历史补充字段，当前 SQL 不返回历史价格，不将它补成现价。公开 DTO 不含内部数据库身份、revision 或查询指纹。

6 Agent V2 : SSE 与安全重放

在同一 POST /messages 请求中设置 stream:true，返回 text/event-stream。其余身份、幂等键和请求约束与 JSON 完全相同；仅修改 Accept 请求头不会开启流式返回。使用 POST fetch 的响应流，不使用仅支持 GET 的原生 EventSource。

事件顺序与 data 结构

status -> state -> (input_required | result) -> [delta] -> done
异常 -> error (终止)

事件	data 的关键字段	语义
status	schema_version、request_id、stage=accepted、message	已接收，不代表工具完成。
state	conversation_id、turn_id、phase、intent、next_action	工作流最终停止态摘要。
input_required	required_inputs	等待输入，包括受控商品候选。
result	result、failure、warnings	完成、失败或 handoff 的投影；result 可能为 null。
delta	content	当前为停止态 reply 的文本投影，不承诺模型逐 Token。
done	schema_version、response	response 是完整 AgentResponse；校验后作为最终快照。
error	code、message、http_status、request_id、retryable 等	流内错误；终止后不会再发送 done。

所有 V2 事件的 schema_version 为字符串 "1"。state / input_required / result / delta 均带 request_id、conversation_id、turn_id；done.response 为完整响应，不使用 RAG 的 metadata / citations 格式。

帧与流内错误示例

```
event: status
data: {"schema_version":"1","request_id":"demo","stage":"accepted",
data: "message":"Agent 已接收请求"}
```

同一事件的多行 data 按换行连接后再解析 JSON；空行结束一个事件。TCP 数据块可能切断 JSON、中文字符或事件边界，不能按每次 read() 直接解析。

例如 error 事件中的 data 可以是：

```
{"schema_version": "1", "request_id": "api-demo-timeout",
"code": "agent_request_timeout", "message": "请求超过服务端时间预算",
"http_status": 504, "category": null,
"retryable": true, "retry_after_seconds": null}
```

断流时不要自动另起请求：HTTP 200 后仍须等待 done / error。提前断开时保留原 payload 与键，由用户选择重试；先清除部分内容再重绘，避免重复追加 delta。重放读取原消息结果。

未实现 Last-Event-ID 偏移续传。浏览器停止读取不保证服务端和上游已经取消。业务返回 phase=failed 仍可正常走 result / delta / done；它与 error 事件所表示的请求失败不是同一层。

7 Assistant V1：兼容单轮查询

此接口由 Assistant 服务提供，直接路径为 POST /v1/chat。当前受控 Agent Web 不代理它；其他可信系统需使用可达的 Assistant 地址和专用服务 Key。它不识别 Agent 会话，不提供候选 Token 恢复，也不接受 history。

请求字段	规则
mode	必填；只允许 policy 或 device_price。由调用方明确选择。
question	必填；去首尾空白后为 1~2000 字符。
stream	可选，默认 true；JSON 必须显式设置 false。额外字段被拒绝。

```
{
  "mode": "device_price",
  "question": "这个设备多少钱？",
  "stream": false
}
```

信息不足的完整响应示例

```
{
  "request_id": "v1-demo-001", "mode": "device_price",
  "answer": "请提供设备品牌、完整型号和必要规格。",
  "evidence": [], "warnings": [],
  "missing_fields": ["brand_or_model"], "used_tool": "device_price",
  "finish_reason": "insufficient_information",
  "reason_code": "", "usage": {}
}
```

finish_reason	处理方式
stop / partial	分别展示完整或部分证据与 warnings；按 evidence.type 渲染。
insufficient_information	提示补充 missing_fields；下一次仍是独立请求，须包含完整问题。
no_match	没有可信证据；不要推测价格或政策。reason_code 可细分拒答门槛。

设备 evidence 包含品牌、型号、规格、字符串 price、currency、source、observed_at，以及原价、售卖状态和 match_score；后者为排序信号，不是准确率。V1 设备包装器可读取统一 catalog_v2 核心，但不提供生鲜模式。

政策证据在不同接口中的位置

接口	回答 / 来源字段
Agent V2 · policy	reply；result.data.evidence / evidence_ids。result.reason_code 说明 stop 或拒答原因。
Assistant V1 · policy	answer；evidence，单项 type=policy、evidence_id；used_tool=policy_knowledge。
RAG /v1/chat	answer；citations，使用 index 与正文 [n] 引用对应。

政策证据保留 title、source_url、excerpt、document_no、published_at、source_org、section_path、chunk_id / document_id 和检索分数；直接展示来源，不从回答反推引用。

V1 SSE 顺序为 status → evidence → delta → [usage] → done；evidence.items 保存证据，done 保存结束原因、warnings 和 missing_fields，不嵌套 AgentResponse。工具错误走 error。没有 V2 的幂等保证；重复调用可能再次访问上游。

8 政策 RAG：直接检索

RAG 是独立服务。使用管理员提供的 RAG Base URL 与 RAG 服务 Key，在 Authorization: Bearer 或 X-API-Key 中二选一发送。不要复用 Assistant 地址或模型供应商 Key。POST /v1/retrieve 返回政策片段，不调用 DeepSeek 生成回答。

请求与筛选

```
{
  "query": "快递业务经营许可条件",
  "top_k": 5, "candidate_k": 40,
  "filters": {
    "document_types": ["html", "pdf"],
    "validity_statuses": ["有效", "unknown"],
    "source_orgs": ["国家邮政局"],
    "published_from": "2018-01-01", "published_through": "2026-09-14"
  }
}
```

参数	默认值与约束（可由服务端收紧）
query	必填；去空白后 1~2000 字符。Embedding Token 超限仍可能返回 422 query_too_long。
top_k	省略或 null 使用配置；代码默认 8，默认服务上限 20，Schema 绝对上限 100。
candidate_k	省略或 null 按配置与 top_k 解析；代码默认 40、服务上限 100、Schema 上限 500；不得小于 top_k。
filters	可省略。类型 / 有效性数组各最多 10 项，发布机构最多 20 项；每项为 1~256 字符。日期为 YYYY-MM-DD，含边界。

同一数组内为 OR，不同字段间为 AND；published_from 不得晚于 published_through。仅提供公开筛选字段，不传 Milvus 原生过滤表达式。

200 响应示例（合成片段）

```
{
  "query": "快递业务经营许可条件", "mode": "hybrid_rrf_rerank",
  "count": 1, "elapsed_ms": 120.5,
  "results": [{
    "rank": 1, "score": 0.032, "rerank_score": 0.91,
    "chunk_id": "demo-chunk", "document_id": "demo-document",
    "parent_document_id": "", "title": "示例政策材料",
    "text": "示例正文，仅用于说明接口结构。",
    "source_url": "https://example.invalid/policy",
    "document_type": "html", "published_at": "2026-01-01 00:00:00",
    "document_no": "", "source_org": "示例机构", "validity_status": "unknown",
    "section_path": "第一章", "chunk_index": 0
  }]
}
```

count 可能小于 top_k，包括 0。mode 为 hybrid_rrf 或 hybrid_rrf_rerank；rank 从 1 开始，rerank_score 未启用重排时为 null。score / rerank_score 是检索信号，不是答案正确率。

elapsed_ms 是该路由计时，包含等待容量及整个检索链路；启用重排时也包含重排，不是单独的 Milvus 耗时。published_at 是来源元数据字符串，可能为空；不要假定所有历史元数据都是完整 ISO 时间。筛选“有效”不替代人工核验政策适用性。

9 政策 RAG：引用式问答与拒答

POST /v1/chat (RAG 服务)

请求使用 question 而非 query；没有 Assistant 的 mode 语义，附带该字段也不会切换业务能力。top_k、candidate_k、filters 与检索相同。stream 默认 true；以下显式选 JSON。回答前先检索、重排、判断证据充分性，再生成引用式文本。

```
{
  "question": "快递业务经营许可需要符合哪些条件？",
  "stream": false, "top_k": 5, "candidate_k": 40,
  "filters": {}
}
```

200 响应示例（合成，不代表政策事实）

```
{
  "request_id": "rag-demo-001", "model": "示例模型标识",
  "answer": "示例回答内容，请结合引用原文核验[1]。",
  "citations": [{
    "index": 1, "chunk_id": "demo-chunk", "document_id": "demo-document",
    "title": "示例政策材料", "source_url": "https://example.invalid/policy",
    "document_no": "", "published_at": "2026-01-01 00:00:00",
    "source_org": "示例机构", "section_path": "第一章",
    "score": 0.032, "rerank_score": 0.91, "excerpt": "示例证据摘要。"
  }],
  "usage": { "prompt_tokens": 100, "completion_tokens": 20, "total_tokens": 120},
  "finish_reason": "stop"
}
```

model 是实际供应商模型标识，不作为客户端分支依据。正文 [n] 对应 citations.index=n；来源 URL、摘要和文档位置应与回答一起呈现。usage 来源于生成模型，可能有扩展字段，不等于整个 Agent 查询的总成本。

finish_reason	含义 / 是否生成回答
stop	正常结束；呈现答案与引用。其他供应商结束值按未完整回答谨慎处理。
no_context	未召回可用上下文；固定资料不足提示，空引用，不调用答案生成。
reranker_rejected	本地重排门槛拒绝；固定提示、空引用，不调用答案生成。
llm_rejected	证据充分性 Judge 拒绝；固定提示、空引用，不再调用答案生成。

RAG SSE 与 Agent SSE 不同

```
metadata -> delta* -> [usage] -> done
error（终止，无 done）
```

metadata 带 request_id、model 和完整 citations；delta.content 按顺序追加；usage 为模型用量；done 带 request_id 与实际 finish_reason。拒答流仍返回 metadata（空引用）、固定 delta 和对应拒答原因的 done，并非一律 no_context。Judge 用量只进入服务监控，不作为答案生成 usage。

忽略：keep-alive 等 SSE 注释，但按空行准确分帧。建立流前检查非 2xx；建立后还须处理 error 和没有终止事件的断流。RAG 不提供 Agent 的 Idempotency-Key 去重语义，重复调用可能再次产生模型费用。

证据门槛异常（例如 Judge 结构无效）是 502 relevance_gate_failed / SSE error，不是正常资料不足。POST /v1/retrieve 只做检索；POST /v1/chat 可能调用 Judge 和生成模型。不要通过关闭门槛来把技术故障变成成功回答。

10 错误处理、健康检查与契约位置

客户端先判断 HTTP，再判断工作流和业务结果。错误响应可能是 detail 对象、FastAPI 422 的 detail 数组，或代理产生的非 JSON；统一保留 HTTP 状态、X-Request-ID 与可解析 code，不依赖中文文案。V2 流内 error 的 http_status 表示原始 HTTP 语义。

```
{
  "detail": {
    "code": "agent_client_upgrade_required",
    "message": "页面版本已更新，请刷新后重试",
    "request_id": "demo-upgrade",
    "retryable": false
  }
}
```

状态 / 常见 code	处理方式
400 / 413 / 422	检查重复安全头、请求大小、类型和未知字段；V2 price_selection_invalid 表示候选失效或作用域不符，不盲重试。
401 / 403	unauthorized、browser_session_*、browser_origin_* 等；修正凭据、核验身份或入口，不伪造 owner。
404 · conversation_not_available	不存在、已删除或非本人会话；不枚举 ID。错误路径也可能由代理返回 404。
409 · browser_session_changed / agent_client_upgrade_required	分别核验新身份或刷新客户端；旧 pending 不自动换身份重发。
409 · conversation_expired / price_query_restart_required	保留可读历史，明确新建查询；不是重复执行旧状态。
409 · idempotency_request_hash_conflict	同键内容不同；先确认是用户新消息还是误改了待重试请求，禁止一律换键。
409 · conversation_run_in_progress / idempotency_request_in_progress	仍在运行；保留原请求，稍后读取状态或受控重试，禁止并行推进。
429 · rate_limit_exceeded	遵守 Retry-After。默认配置为每 Key 每 60 秒 60 次；浏览器共享代理额度，reset 不能绕过。
502 / 503 / 504	分别常见于契约 / 上游错误、依赖 / 状态不可用、超时；仅在允许重试时有限退避，V2 保留原键。

V1 典型依赖错误：tool_failed / tool_contract_failed (502)、tool_unavailable (503)。RAG: retrieval_failed / chat_provider_failed / relevance_gate_failed (502)；*_unavailable (503)。遇到 500 或未知 code 应保留证据排查。HTTP 200 + phase=failed 与正常 no_match 必须区分。

运维与服务间端点：不经当前 Web 入口

两服务 GET /health/live 仅表进程，GET /health/ready 表依赖。Assistant 根 health 是 V1 视角；memory=disabled 不能据此判断 V2。

Assistant GET /v2/agent/health/ready 检查状态库、checkpoint 与能力；关键项不就绪为 503，200 + degraded 仍要查看 checks。不要求未装配的三物流 available。

RAG GET /v1/auth/check 只核验 Key，返回 {status:ok, service:spb-rag-api}；不执行检索或模型。

健康端点不要求业务 Key，仍只应在运维网络开放；/metrics 条件启用，同样不面向浏览器。/docs、/openapi.json 直连服务时受 Key 保护，当前 Web 不代理。readiness 不能证明具体问题可回答、存在某型号或数据为实时。

契约入口：Agent V2 OpenAPI · 生成 TypeScript · Assistant DTO · RAG DTO。同目录 routes 为对应 HTTP 实现。字段更改时应同步 DTO、OpenAPI、客户端类型与运行时校验。